

MongoDB

What is MongoDB?

MongoDB is:

A NoSQL document database.

Instead of tables & rows (like MySQL),
it stores:

- Collections → like tables
- Documents → like rows
- Stored in JSON-like format (BSON)

Example document:

```
{  
  "name": "Prajwal",  
  "age": 22,  
  "isAdmin": false  
}
```

SQL vs MongoDB

SQL	MongoDB
Table	Collection
Row	Document
Column	Field
Fixed schema	Flexible schema

🔥 Why MongoDB is popular in MERN?

- JSON everywhere (JS backend + frontend)

- Flexible structure
- Scales horizontally
- Fast prototyping

2 What is Mongoose?

MongoDB by itself is just a database.

Mongoose is:

👉 An ODM (Object Data Modeling) library for Node.js

It sits between:

Node.js app → Mongoose → MongoDB

Why we need Mongoose?

Because MongoDB is flexible.

Too flexible.

Without structure:

- Anyone can insert random fields
- No validation
- No relationships control

Mongoose gives:

- Schema
- Validation
- Models
- Middleware
- Cleaner syntax

What is a Schema?

A Schema defines structure of document

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  password: String,
  isAdmin: {
    type: Boolean,
    default: false
  }
});
```

Now we define rules.

What is a Model?

Model = compiled version of schema.

```
const User = mongoose.model("User", userSchema);
```

Now you can:

```
User.create()
User.find()
User.findById()
User.deleteOne()
```

So:

Schema → Blueprint

Model → Actual usable class

Document → Real data instance

5 Folder Structure (Professional Way)

Since you want to build strong backend:

```
project/
|
|— models/
|   └─ user.model.js
|
|— config/
|   └─ db.js
|
|— routes/
|— controllers/
```

Clean architecture mindset already.

6 Connect MongoDB to Node (Step-by-Step)

Step 1: Install

```
npm install mongoose
```

Step 2: Create DB connection

config/db.js

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect("mongodb://127.0.0.1:27017/myapp");
    console.log("MongoDB connected");
  } catch (error) {
    console.error(error);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Step 3: Use in server.js

```
const express = require("express");
const connectDB = require("../config/db");

const app = express();

connectDB();

app.listen(5000);
```

Create User Model

models/user.model.js

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true,
    unique: true
  },
  password: {
    type: String,
    required: true
  }
}, { timestamps: true });

module.exports = mongoose.model("User", userSchema);
```

Notice:

- required
- unique
- timestamps

This is professional backend thinking.

Create User (Example Controller)

```
const User = require("../models/user.model");
```

```
const createUser = async (req, res) => {  
  try {  
    const user = await User.create(req.body);  
    res.json(user);  
  } catch (err) {  
    res.status(400).json({ error: err.message });  
  }  
}
```

Advanced Querying

What Is a Mongoose Query?

When you write:

```
User.find({ age: 18 })
```

You are saying:

“Go inside the users collection and give me documents where age = 18”

That’s it.

Mongoose queries are just JavaScript objects that describe conditions.

Basic Filtering (The Right Way to Understand It)

Assume this document exists:

```
{  
  name: "Prajwal",  
  age: 22,  
  role: "admin",  
  isActive: true  
}
```

```
User.find({ role: "admin" })
```

Meaning:

Give me all users whose role is exactly "admin"

◇ Greater Than / Less Than

```
User.find({ age: { $gt: 18 } })
```

Meaning:

Give users whose age is greater than 18

Operators:

Operator	Meaning
\$gt	greater than
\$gte	greater than equal
\$lt	less than
\$lte	less than equal

Operator	Meaning
<code>\$ne</code>	not equal
<code>\$in</code>	inside array

Example:

```
User.find({ age: { $in: [20, 22, 25] } })
```

Meaning:

Age must be either 20, 22 or 25

2 AND / OR Conditions

AND (both must be true)

```
User.find({  
  age: { $gt: 18 },  
  isActive: true  
})
```

Meaning:

Age > 18 AND user is active

OR (either can be true)

```
User.find({  
  $or: [  
    { role: "admin" },  
    { age: { $gt: 60 } }  
  ]  
})
```

Meaning:

Role is admin OR age > 60

3 Sorting

`User.find().sort({ age: 1 }) // show the youngest user first`

`User.find().sort({ age: -1 }) // show the oldest user first`

1 = ascending

-1 = descending

4 Limiting Results

`User.find().limit(5)`

Meaning:

Only give me 5 users

Pagination (Page System)

`const page = 2;`

`const limit = 5;`

`User.find()`

`.skip((page - 1) * limit)`

`.limit(limit);`

If page = 2

skip = 5

Meaning:

Skip first 5 users, then give next 5

6 Selecting Fields (Projection)

`User.find().select("name email")`

Meaning:

Only return name and email

Exclude password:

```
User.find().select("-password")
```

7 Updating Data

Update One

```
User.updateOne(  
  { email: "test@gmail.com" },  
  { role: "admin" }  
)
```

Meaning:

Find user with this email and change role to admin

Increment Field

```
User.updateOne(  
  { _id: userId },  
  { $inc: { balance: 100 } }  
)
```

Meaning:

Increase balance by 100

Decrease balance by 100

```
User.updateOne(  
  { _id: userId },  
  { $inc: { balance: -100 } }  
);
```

\$inc with a negative number.

8 Delete

```
User.deleteOne({ email: "test@gmail.com" })
```

Meaning:

Delete user with that email

What Is Indexing? (Simple Example)

Imagine this:

You have 10 lakh (1,000,000) users in database.

Now you run:

```
User.find({ email: "abc@gmail.com" })
```

Without index:

MongoDB checks:

- User 1
- User 2
- User 3
- ...
- Until it finds match

This is called:

Full collection scan

Slow.

Now with index on email:

MongoDB keeps a separate sorted list like:

abc@gmail.com → userId1

hello@gmail.com → userId2

xyz@gmail.com → userId3

Now it directly jumps to that email.

Fast.

How To Create Index in Mongoose

Method 1 (Simple)

```
const userSchema = new mongoose.Schema({  
  email: {  
    type: String,  
    unique: true  
  }  
});
```

unique: true automatically creates index.

Method 2 (Manual Index)

```
userSchema.index({ email: 1 });
```

1 = ascending

-1 = descending

What Fields Should Be Indexed?

Index fields that you:

- **Frequently search**
- **Frequently filter**
- **Frequently sort**
- **Use in login (email, username)**
- **Use as foreign keys**

Example:

```
User.find({ email: "test@gmail.com" })
```

→ index email

What NOT To Index

Do NOT index:

- Boolean fields (isActive)
- Fields rarely searched
- Very frequently updated fields

Why?

Because:

Every time you insert or update:

MongoDB must also update index.

More indexes = slower writes.

Compound Index (Very Important)

If you do queries like:

```
User.find({ role: "admin", isActive: true })
```

Better index:

```
userSchema.index({ role: 1, isActive: 1 });
```

This is called compound index.

Order matters.

When Index Is NOT Used

If you search like:

```
User.find({ name: { $regex: "raj" } })
```

Index may not work properly.

Also:

If you filter on non-indexed field → full scan.

Check If Index Exists

In MongoDB shell:

```
db.users.getIndexes()
```

Very Important Concept

Index improves:

✓ Read speed

But reduces:

✗ Write speed

✗ Storage space

So balance is important.

Indexing in MongoDB uses B-tree data structures to improve read performance. It avoids full collection scans by allowing efficient lookup of documents. However, indexes increase write cost and storage usage, so they must be created strategically based on query patterns. Compound indexes must follow the left-prefix rule to be effective.

With an index, MongoDB uses a B-tree structure to directly locate matching documents in logarithmic time ($O(\log n)$) instead of linear time ($O(n)$).

Why Do We Need Indexing?

In real-world applications:

- Login systems search by email
- E-commerce apps filter by category
- Dashboards sort by createdAt
- Orders are fetched by userId

If these fields are not indexed:

- Queries become slow as data grows
- CPU usage increases
- Response time increases

Indexes solve this scalability problem.

Types of Indexes (Important for Interviews)

1 Single Field Index

```
userSchema.index({ email: 1 });
```

Used when filtering by one field frequently.

2 Compound Index

```
productSchema.index({ category: 1, price: 1 });
```

Used when:

- You filter by one field
- And sort by another

Very common in production.

Order matters.

3 Unique Index

```
email: { type: String, unique: true }
```

Ensures no duplicate values.

Used in:

- Emails
- Usernames

4 Text Index

Used for search functionality.

✅ Tradeoffs of Indexing (Very Important)

Indexes improve:

✓ Read performance

But they reduce:

✗ Write performance

✗ Increase storage

✗ Increase RAM usage

Because every insert/update must also update the index.

So over-indexing hurts performance.

REGEX

pattern matching

It allows you to search text partially instead of exact match.

Instead of:

```
User.find({ name: "Prajwal" })
```

You can search:

- Names that start with "Pra"
- Names that contain "raj"
- Case-insensitive search

Basic Regex Search in Mongoose

Search names containing "raj"

```
User.find({  
  name: { $regex: "raj", $options: "i" }  
});
```

What this means:

- \$regex → pattern to match

- \$options: "i" → case insensitive

This will match:

- Prajwal
- RAJ
- Rajesh
- praj

Important Regex Patterns

◆ Starts with

```
User.find({  
  name: { $regex: "^pra", $options: "i" }  
});
```

^ means start of string.

Matches:

- Prajwal
- Pratik

Ends with

```
User.find({  
  name: { $regex: "wal$", $options: "i" }  
});
```

\$ means end of string.

Matches:

- Prajwal
- Rahulwal

Contains (most common)

```
User.find({  
  name: { $regex: "raj", $options: "i" }  
});
```

Matches anywhere inside string.

What Is a Session?

A **session** is like a container that groups multiple database operations together.

Think of it like this:

“I want these operations to behave as one single unit.”

That’s it.

Session alone does nothing special.

It becomes powerful when you use it with a **transaction**.

What Is a Transaction?

A transaction means:

Either all operations succeed

OR none of them succeed

Example: Money Transfer

You must:

1. Deduct ₹100 from User A
2. Add ₹100 to User B

If step 2 fails, step 1 must be undone.

Otherwise money disappears.

Simple Mongoose Transaction Example

```
const session = await mongoose.startSession();

session.startTransaction();

try {
  await User.updateOne(
    { _id: senderId },
    { $inc: { balance: -100 } },
    { session }
  );

  await User.updateOne(
    { _id: receiverId },
    { $inc: { balance: 100 } },
    { session }
  );

  await session.commitTransaction();
  session.endSession();

} catch (error) {
  await session.abortTransaction();
  session.endSession();
}
```

What Is Actually Happening Internally?

When transaction starts:

- MongoDB creates a snapshot
- Changes are temporary
- Other users cannot see uncommitted changes

If commit:

→ Changes become permanent

If abort:

→ Everything rolls back

Is This Locking?

Yes, but not like SQL.

MongoDB uses:

Document-level locking

It does NOT lock entire table.

It only locks the documents being modified.

So other documents remain accessible.

When Do You Need Session + Transaction?

Use it when:

- ✓ Updating multiple documents
- ✓ Financial operations
- ✓ Booking system (seat reservation)
- ✓ Inventory + order creation together

When You DO NOT Need It

If you are updating only ONE document:

```
User.updateOne({ _id }, { $inc: { balance: -100 } });
```

You do NOT need transaction.

Because single document updates are already atomic.

In MongoDB, sessions are used to implement transactions. Transactions ensure multiple operations across documents either fully succeed or fully fail.

MongoDB uses document-level locking and snapshot isolation instead of traditional row-level locking.